

Система Диффи–Хеллмана

Эта криптосистема была открыта в середине 70-х годов американскими учеными:

- **Мartiном Хеллманом**
- **Уитфилдом Диффи**

Это первая система, которая позволяла защищать информацию без использования секретных ключей, передаваемых по защищенным каналам.



Мартин Хеллман, Уитфилд Диффи

Схема Диффи-Хеллмана

Строится система для нескольких абонентов: А, В, С.

Общие открытые параметры системы:

p – большое простое число,

g – первообразный корень по модулю p , $1 < g < p - 1$

Секретные параметры абонентов:

X_A, X_B, X_C – большие случайные числа в диапазоне $[2; p - 1]$

Открытые параметры абонентов:

$$Y_A = g^{X_A} \bmod p,$$

$$Y_B = g^{X_B} \bmod p,$$

$$Y_C = g^{X_C} \bmod p.$$

Задача быстрого нахождения чисел p и g

При произвольно заданном p она может оказаться трудной задачей, связанной с разложением на простые множители числа $p-1$.

Для обеспечения высокой стойкости рассмотренной системы число $p-1$ должно обязательно содержать большой простой множитель (в противном случае можно быстро вычислить дискретный логарифм с помощью алгоритма Полига–Хеллмана).

Поэтому часто рекомендуют использовать следующий подход. Простое число p выбирается таким, чтобы выполнялось равенство

$$p = 2q + 1,$$

где q — также простое число. Тогда в качестве g можно взять любое число, для которого справедливы неравенства

$$1 < g < p - 1 \quad \text{и} \quad g^q \bmod p \neq 1.$$

При работе алгоритма каждая сторона:

1. Генерирует случайное натуральное число — закрытый ключ
2. Совместно с удалённой стороной устанавливает открытые параметры **p** и **g** (обычно значения **p** и **g** генерируются на одной стороне и передаются другой), где **p** является случайным простым числом, а **q = (p-1)/2** также должно быть случайным простым числом (для повышения безопасности).
g является первообразным корнем по модулю p (также является простым числом).
3. Вычисляет открытый ключ, используя преобразование над закрытым ключом

$$Y_A = g^{X_A} \bmod p$$

$$Y_B = g^{X_B} \bmod p$$

4. Обменивается открытыми ключами с удалённой стороной
5. Вычисляет общий секретный ключ **Z**, используя открытый ключ удаленной стороны В и свой закрытый ключ.

На стороне А:

$$\begin{aligned} Z_{AB} &= Y_B^{X_A} \mod p = \\ &= (g^{X_B} \mod p)^{X_A} \mod p = \\ &= (g^{X_B})^{X_A} \mod p = \\ &= g^{X_B X_A} \mod p \end{aligned}$$

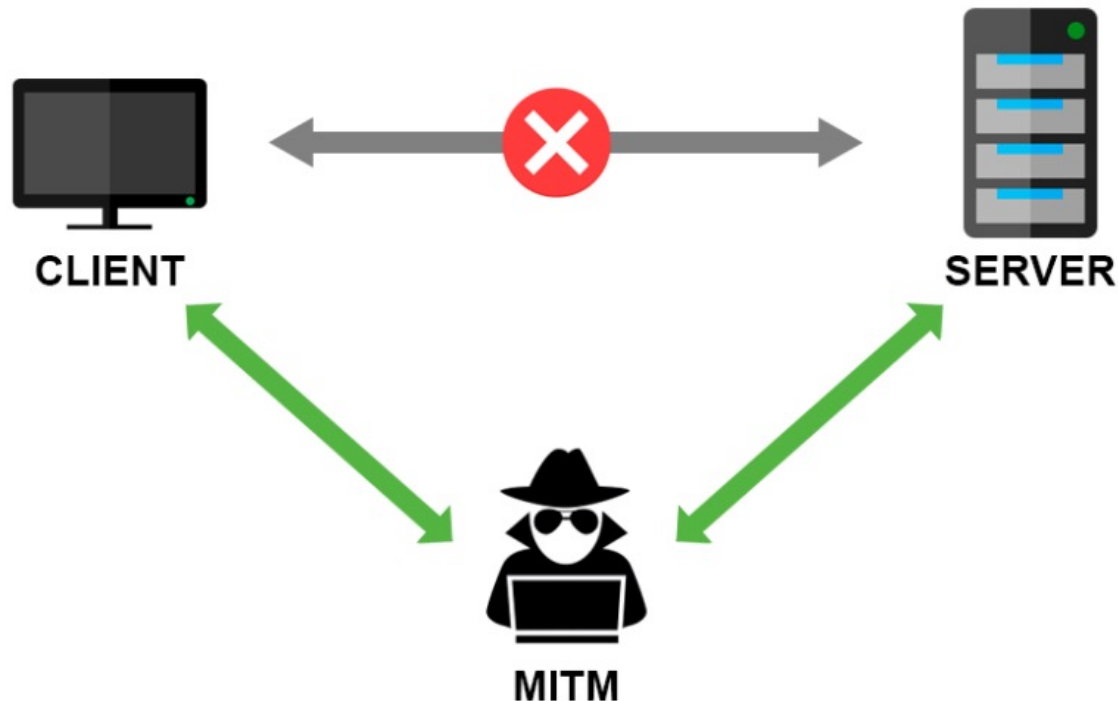
На стороне В:

$$\begin{aligned} Z_{BA} &= Y_A^{X_B} \mod p = \\ &= (g^{X_A} \mod p)^{X_B} \mod p = \\ &= (g^{X_A})^{X_B} \mod p = \\ &= g^{X_A X_B} \mod p \end{aligned}$$

Атака посредника - Man in the middle (MITM)

Злоумышленник тайно ретранслирует и при необходимости изменяет связь между двумя сторонами, которые считают, что они непосредственно общаются друг с другом.

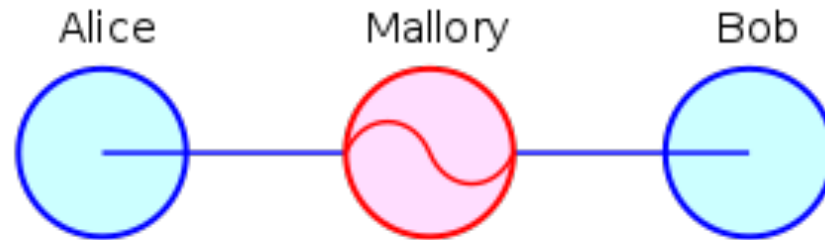
Взломщик осуществляет вмешательство в протокол передачи, **удаляя или искажая информацию.**



Данная атака направлена на обход взаимной аутентификации или отсутствие таковой.

Она будет успешной только тогда, когда злоумышленник имеет возможности:

- выдать себя за каждую конечную точку
- или оставаться незамеченным в качестве промежуточного узла.



Пример атаки на алгоритмическом языке

1. Алиса отправляет Бобу сообщение, которое перехватывает Мэлори:

Алиса Боб *«Привет, Боб, это Алиса. Пришли мне свой открытый ключ.»* → Мэлори

2. Мэлори пересылает сообщение Бобу; Боб не может догадаться, что это сообщение не от Алисы:

Алиса Мэлори Боб *«Привет, Боб, это Алиса. Пришли мне свой открытый ключ.»* →

3. Боб посылает свой ключ:

Алиса Мэлори ← [ключ Боба] Боб

4. Мэлори подменяет ключ Боба своим и пересылает сообщение Алисе:

Алиса ← [ключ Мэлори] Мэлори Боб

5. Алиса шифрует сообщение ключом Мэлори, считая, что это ключ Боба, и только он может расшифровать его:

Алиса «Встречаемся на автобусной остановке!» [зашифровано ключом Мэлори]
→ Мэлори Боб

6. Мэлори расшифровывает сообщение, читает его, модифицирует его, шифрует ключом Боба и отправляет его:

Алиса Мэлори «Жди меня у входа в музей в 18:00.» [зашифровано ключом Боба] →
Боб

7. Боб считает, что это сообщение Алисы.

Атака на протокол Диффи-Хеллмана

Рассмотрим атаку на протокол выработки общего секрета Диффи-Хеллмана между сторонами **A** и **B**. Допустим, криптоаналитик **E** имеет возможность не только перехватывать сообщения, но и подменять их своими, то есть осуществлять активную атаку:

$$A \leftrightarrow E \leftrightarrow B.$$

Перехват и подмена ключей

1. Сторона **A** отправляет сообщение стороне **B**:

$$A \xrightarrow{E} B : g^x \bmod p.$$

2. Криптоаналитик **E** перехватывает сообщение стороны **A** и подменяет его, отправляя стороне **B** уже другое сообщение :

$$E \rightarrow B : g^z \bmod p.$$

3. Сторона **B** отправляет сообщение стороне **A**:

$$A \xleftarrow{E} B : g^y \bmod p.$$

4. Криптоаналитик **E** перехватывает сообщение стороны **B** и подменяет его, отправляя стороне **A** какое-то своё сообщение :

$$A \leftarrow E : g^z \bmod p.$$

5. Результатом данных действий является образование двух каналов связи криптоаналитика **E** со сторонами **A** и **B**, причем сторона **A** считает что общается со стороной **B** при помощи секретного ключа K_{AE} , а сторона **B** отправляет сообщения при помощи ключа K_{BE} . При этом стороны **A** и **B** не подозревают, что обмен сообщениями происходит не напрямую, а через криптоаналитика **E**:

$$\begin{aligned}K_{AE} &= g^{xz} \bmod p, \\K_{BE} &= g^{yz} \bmod p.\end{aligned}$$

Подмена сообщений

1. Сторона **A** отправляет сообщение **m** стороне **B**, зашифрованное при помощи ключа K_{AE} :

$$A \xrightarrow{E} B : E_{K_{AE}}(m).$$

2. Криптоаналитик **E** перехватывает это сообщение, расшифровывает ключом K_{AE} , при необходимости, изменяет его на m' , зашифровывает ключом K_{BE} и отправляет стороне **B**: K_{AE} :

$$E \rightarrow B : E_{K_{BE}}(m').$$

3. Аналогичные действия криптоаналитик **E** предпринимает при передачи сообщений от **B** к **A**.

Сценарии атаки

- Атака на системы с открытым ключом
- Внедрение вредоносного кода
- Downgrade Attack
- Публичные средства коммуникации

Обнаружение MITM-атаки

- IP-адрес сервера
- DNS-сервер
- X.509-сертификат сервера
 - Подписан ли сертификат самостоятельно?
 - Подписан ли сертификат центром сертификации?
 - Был ли сертификат аннулирован?
 - Менялся ли сертификат недавно?
 - Получали ли другие клиенты в интернете такой же сертификат?

Способы нахождения простых чисел

«Решето Эратосфена»



Древнегреческий математик Эратосфен, живший более чем за 200 лет до н.э., составил **первую таблицу простых чисел**, которая получила название «Решето Эратосфена».

Согласно легенде, Эратосфен писал числа на дощечке, покрытой воском, и прокалывал дырочки в тех местах, где были написаны составные числа.

Поэтому дощечка являлась неким подобием решета, через которое «просеивались» все составные числа, а оставались только числа простые. Эратосфен сформировал таблицу простых чисел до 1000.

Сложность алгоритма составляет **$O(n * \log(\log n))$** операций при составлении таблицы простых чисел до n .

Выпишем несколько подряд идущих чисел, начиная с 2. Двойку обведем как простое число, а остальные числа, кратные 2, зачеркнем. Ближайшим незачёркнутым числом будет 3. Возьмём в коллекцию и его, а все остальные числа, кратные 3, зачеркнем.

При этом окажется, что некоторые числа уже были вычеркнуты раньше, как, например, 6, 12 и др. Следующее наименьшее незачёркнутое число – это 5.

Берем 5, а остальные числа, кратные пятерке, зачеркиваем. Повторяя эту процедуру снова и снова, добьемся того, что незачеркнутыми будут одни лишь простые числа (на рисунке они обведены в кружок).

	2	3	4	5	6	7	8	9	10		Prime numbers				
11	12	13	14	15	16	17	18	19	20			2	3	5	7
21	22	23	24	25	26	27	28	29	30			11	13	17	19
31	32	33	34	35	36	37	38	39	40			23	29	31	37
41	42	43	44	45	46	47	48	49	50			41	43	47	53
51	52	53	54	55	56	57	58	59	60			59	61	67	71
61	62	63	64	65	66	67	68	69	70			73	79	83	89
71	72	73	74	75	76	77	78	79	80			97	101	103	107
81	82	83	84	85	86	87	88	89	90			109	113		
91	92	93	94	95	96	97	98	99	100						
101	102	103	104	105	106	107	108	109	110						
111	112	113	114	115	116	117	118	119	120						

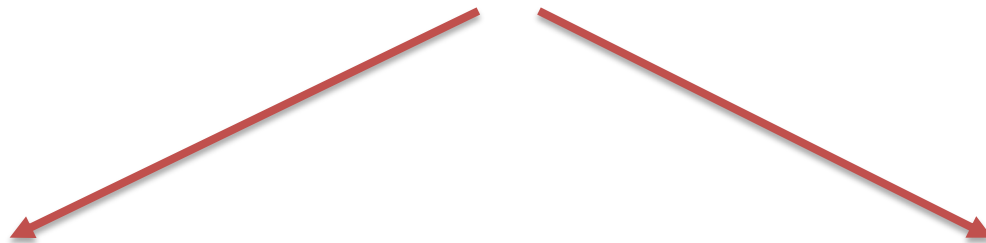
Другие способы нахождения простых чисел

На практике вместо получения списка простых чисел зачастую требуется проверить, является ли данное число простым.

Например, для шифрования используются большие простые числа и возникает необходимость проверки числа на простоту.

Алгоритмы, решающие эту задачу, называются **тестами простоты**.

Тесты простоты (тесты на простоту числа)



Универсальные

для любых чисел

- Тест Ферма
- Тест Миллера-Рабина
- Тест Соловея-Штрассена
- и т.д.

Специализированные

для определенного класса чисел

- Тест Люка-Лемера для чисел Мерсенна вида:
$$M_n = 2^n - 1,$$
где n — простое число
- и т.д.

Различают **полиномиальные**, **детерминированные** и **вероятностные** тесты простоты.

Полиномиальность означает, что наибольшее время работы алгоритма ограничено многочленом от количества цифр в проверяемом числе.

Детерминизм гарантирует получение уникального предопределённого результата.

Вероятностные тесты могут проверить простоту числа за полиномиальное время, но при этом дают лишь вероятностный ответ.

Проверка числа на простоту за $O(\sqrt{P})$

Чтобы определить, является ли данное число P простым, безусловно, достаточно написать простой цикл поиска делителей числа P :

```
bool prime (long long p){  
    for (long long i = 2; i <= sqrt(p); i++)  
        if (p%i == 0)  
            return false;  
    return true;  
}
```

Данная функция проверки числа на простоту достаточно эффективна. Однако иногда нужно уметь проверять число на простоту быстрее.

Проверка числа на простоту за $O(K \cdot \log P)$

Чтобы проверить число **P** на простоту с достаточно хорошей вероятностью безошибочности, достаточно **K** раз (например, 100 раз) проверить случайное число **a** тестом Ферма.

Теорема 2.6 (Ферма). Пусть p — простое число и $0 < a < p$.
Тогда

$$a^{p-1} \bmod p = 1.$$

1. Если для основания a и модуля n выполняется $a^{n-1} \equiv 1 \pmod{n}$, то n может быть простым числом.
2. Если $a^{n-1} \not\equiv 1 \pmod{n}$, то n — однозначно составное.

При проверке числа на простоту тестом Ферма выбирают несколько чисел **a**. Чем больше количество **a**, для которых $a^{n-1} \bmod p = 1$, тем больше шансы, что число **p** - простое.

Однако существуют составные числа, для которых сравнение $a^{n-1} \bmod p = 1$ выполняется для всех **a**, взаимно простых с **p**.

Это числа Кармайкла, их бесконечное множество, наименьшее число Кармайкла — 561.

Число Кармайкла - это составное число **p**, такое что $a^{n-1} \bmod p = 1$ для всех целых чисел **a**, взаимно простых с **p**.

Примеры чисел Кармайкла:

**561, 1105, 1729, 2465, 2821, 6601, 8911, 10585, 15841, 29341,
41041, 46657, 52633, 62745, 63973, 75361, ...**

**Тем не менее, тест Ферма довольно эффективен для
обнаружения составных чисел.**

Тест Ферма эффективен для обнаружения составных чисел при верном выборе параметра **K** .

Данный параметр означает количество различных чисел **a** из диапазона **$0 < a < p$** , которые необходимо перебрать для проверки числа **p** на простоту.

$$P \text{ (число простое)} = 1 - \frac{1}{2^K},$$

где **K** — количество чисел **a** (кол — во итераций проверки).

$$\text{При } K = 1: P \text{ (число простое)} = 1 - \frac{1}{2^1} = \frac{1}{2}$$

$$\text{При } K = 2: P \text{ (число простое)} = 1 - \frac{1}{2^2} = \frac{3}{4}$$

$$\text{При } K = 10: P \text{ (число простое)} = 1 - \frac{1}{2^{10}} = \frac{1023}{1024} \approx 1$$

Теорема 2.6 (Ферма). Пусть p — простое число и $0 < a < p$.
Тогда

$$a^{p-1} \bmod p = 1.$$

```
bool ferma (long long p){  
    if (p == 2)  
        return true;  
    srand (time (NULL));  
    for (int i = 0; i < 100; i++){  
        long long a = (rand() % (p - 2)) + 2;  
        if (gcd (a, p) != 1)  
            return false;  
        if (pows (a, p-1, p) != 1)  
            return false;  
    }  
    return true;  
}
```

Числа **a** и **p** должны быть взаимно просты. Если это условие не выполняется, то число **p** — заведомо непростое.

Данная функция проверки использует функции нахождения НОД, а также быстрого возведения в степень по модулю.

Тест Ферма достаточно эффективен для обнаружения составных чисел при верном выборе параметра ***K***.

Однако числа Кармайкла успешно проходят тест Ферма, хотя не являются простыми.

Отчасти их можно отсеять за счёт проверки наибольшего общего делителя, но такая проверка не имеет хорошего теоретического обоснования вероятности отсеечения, поэтому необходимо рассмотреть и другие тесты.

Тест Миллера-Рабина

Алгоритм был разработан Гари Миллером в 1967 году и модифицирован Майклом Рабином в 1980 году.

Это вероятностный полиномиальный тест простоты. Он позволяет эффективно определять, является ли данное число составным.

Однако с его помощью нельзя строго доказать простоту числа. Тем не менее тест Миллера — Рабина часто используется в криптографии для получения больших случайных простых чисел.

После выполнения **K** раундов теста, мы можем утверждать, что число составное с вероятностью не более $\frac{1}{4^K}$.

Алгоритм Миллера — Рабина параметризуется количеством раундов **K**.

Рекомендуется брать **K** порядка величины **$\log(p)$** , где **p** — проверяемое число.

Ввод: $n > 2$, нечётное натуральное число, которое необходимо проверить на простоту;
 k — количество раундов.

Вывод: *составное*, означает, что n является составным числом;
вероятно простое, означает, что n с высокой вероятностью является простым числом.

Представить $n - 1$ в виде $2^s \cdot t$, где t нечётно, можно сделать последовательным делением $n - 1$ на 2.

цикл A: повторить k раз:

Выбрать случайное целое число a в отрезке $[2, n - 2]$

$x \leftarrow a^t \bmod n$, вычисляется с помощью алгоритма возведения в степень по модулю

если $x = 1$ или $x = n - 1$, то перейти на следующую итерацию цикла A

цикл B: повторить $s - 1$ раз

$x \leftarrow x^2 \bmod n$

если $x = 1$, то вернуть составное

если $x = n - 1$, то перейти на следующую итерацию цикла A

вернуть составное

вернуть *вероятно простое*

Тест Соловея – Штрассена

Это вероятностный тест простоты, открытый в 1970-х годах Робертом Мартином Соловеем совместно с Фолькером Штрассеном.

Тест всегда корректно определяет, что простое число является простым, но для составных чисел с некоторой вероятностью он может дать неверный ответ.

Данный тест опирается на малую теорему Ферма и свойства символа Якоби.

Теорема: если n – нечетное составное число, то количество целых чисел a , взаимно простых с n и меньших n , удовлетворяющих сравнению $a^{(n-1)/2} \equiv \left(\frac{a}{n}\right) \pmod{n}$, не превосходит $n/2$. (*)

$\left(\frac{a}{n}\right)$ - символ Якоби.

Сначала для алгоритма выбирается целое число $k \geq 1$. Тест проверки простоты числа n состоит из k отдельных раундов. В каждом раунде выполняются следующие действия:

1. Случайным образом выбирается число $a < n$, и вычисляется $d = \text{НОД}(a, n)$.
2. Если $d > 1$, то выносится решение о том, что n – составное. Иначе проверяется сравнение (*). Если оно не выполнено, то n – составное. Иначе a является свидетелем простоты числа n .
3. Если после завершения k раундов найдено k свидетелей простоты, то делаем заключение « n – вероятно простое число».

Пример.

Проверим число $n = 19$ на простоту.

Выберем параметр точности $k = 2$.

Решение:

$K=1$. Выберем случайное число $a = 11$.

Проверим условие $\text{НОД}(a,n) > 1$. $\text{НОД}(11,19)=1$. Значит проверяем выполнение сравнения $a^{(n-1)/2} \equiv \left(\frac{a}{n}\right) \pmod{n}$.

$$r = \left(\frac{a}{n}\right) = \left(\frac{11}{19}\right) = 1. \quad s = a^{(n-1)/2} = 11^{(19-1)/2} \pmod{19} = 1.$$

Получили, что $r = s$. Поэтому переходим к следующей итерации.

$K=2$. Выберем случайное число $a = 5$.

Проверим условие $\text{НОД}(a,n) > 1$. $\text{НОД}(5,19)=1$. Значит проверяем выполнение сравнения $a^{(n-1)/2} \equiv \left(\frac{a}{n}\right) \pmod{n}$.

$$r = \left(\frac{a}{n}\right) = \left(\frac{5}{19}\right) = 1. \quad s = a^{(n-1)/2} = 5^{(19-1)/2} \pmod{19} = 1.$$

Получили, что $r = s$. Это была последняя итерация. **Вывод:** 19 – простое число.

Тест Люка – Лемера

Тест был предложен французским математиком **Франсуа Эдуардом Люка** в 1878 году и дополнен американским математиком **Дерриком Генри Лемером** в 1930 году.

Данный тест является полиномиальным, детерминированным и носит имя двух ученых, которые совместно открыли его, даже не встречаясь при жизни.

Тест Люка - Лемера — эффективный тест простоты для чисел Мерсенна. Благодаря этому тесту самыми большими известными простыми числами всегда были простые числа Мерсенна, причём даже до появления компьютеров.

Числа Мерсенна: $M_q = 2^q - 1$, где q — простое число.

Первые числа Мерсенна: 1, 3, 7, 15, 31, 63, 127, 255, 511, 1023, 2047, 4095, 8191, 16 383, 32 767, 65 535, 131 071, ...

Тест базируется на наблюдении, что простота числа мерсенна M_q влечет простоту числа q , а также на утверждении: пусть q — простое число, причем $q \geq 3$.

Определим последовательность L_n следующим образом:

$$L_0 = 4, L_{n+1} = L_n^2 - 2.$$

Тогда M_q - простое тогда и только тогда, когда $L_{q-2} \bmod M_q = 0$.

Возможными значениями L_0 являются: 4, 10, 52, 724, 970, 10084...

Тест Адлемана – Померанса – Румели

В начале 80-х годов Леонард Адлеман, Карл Померанс и Роберт Румели предложили детерминированный алгоритм проверки простоты чисел. Для заданного натурального числа **n** алгоритм выдает верный ответ, **составное n** или **простое**.

Этот алгоритм оказался непрактичным и довольно сложным для реализации на компьютере.

Существенные теоретические упрощения алгоритма Адлемана – Померанса – Румели были получены Х. Ленстрой.

Реализация этого алгоритма позволила проверять на простоту числа **n** порядка 10^{100} за несколько минут.

Алгоритм Ленстры – Коена

Дальнейшие усовершенствования алгоритма Адлемана - Померанса - Румели и алгоритма Х. Ленстры были предложены Хендриком Ленстрой и Генри Коэном.

Однако на практике он оказался наиболее эффективным. При правильной организации его работы алгоритм всегда достаточно быстро выдает правильный ответ, **простое p или составное.**

Алгоритм проверяет на простоту числа порядка $10^{100} - 10^{200}$ за несколько минут.

Тест Агравала – Каяла – Саксены (AKS)

В 2004 г. индийскими учеными Маниндрой Агравалом и его двумя студентами Нираджем Каялом и Нитином Саксеной Индийского технического института Канпура был разработан полиномиальный детерминированный тест простоты.

За свое открытие авторы получили премию Гёделя и приз Фалькерсона в 2006 году.

Это единственный известный на данный момент универсальный (то есть применимый ко всем числам) полиномиальный, детерминированный и безусловный (то есть не зависящий от недоказанных гипотез) тест простоты чисел, основанный на обобщении малой теоремы Ферма на многочлены.